

Math Nuggets for GEN AI

Preference Modeling: Mathematical Foundations of the Bradley-Terry Model for RLHF

By Dr. Anish Roychowdhury, Plaksha University

October 3, 2025

From Absolute Scores to Relative Preferences

While traditional supervised learning uses absolute labels (correct/incorrect, numerical scores), human evaluation of language model outputs is inherently comparative. People find it easier to say "Response A is better than Response B" than to assign absolute quality scores. Preference modeling provides a mathematical framework to convert these relative judgments into scalar reward signals.

Consider the task: "Train a reward model to predict which of two chatbot responses humans would prefer, based on thousands of pairwise comparisons."

Traditional scoring approaches face limitations for subjective quality assessment:

- **Absolute Calibration:** Different annotators may use different scales for "good" vs "excellent"
- **Context Dependence:** Quality depends heavily on the prompt and desired style
- **Cognitive Difficulty:** Assigning numerical scores is harder than making comparisons
- **Inconsistency:** Absolute ratings are more prone to variance across annotators

Why Preference Modeling?

Preference modeling addresses these challenges by learning from relative comparisons, which are more reliable and natural for human annotators. The Bradley-Terry model provides an elegant probabilistic framework for this task.

The Bradley-Terry Model: Mathematical Framework

The Bradley-Terry model, introduced in 1952, provides a probabilistic framework for modeling pairwise comparisons. In the context of RLHF, it converts human preferences into a learnable reward function.

Core Mathematical Setup:

Given a prompt x and two responses y_w (preferred/winner) and y_l (dispreferred/loser), we want to model:

$$P(y_w \succ y_l | x) \tag{1}$$

where $y_w \succ y_l$ denotes "response y_w is preferred over y_l ."

The Bradley-Terry Assumption:

The model assumes that each response has an underlying "strength" or "quality" score $r(y|x)$, and the probability of preferring one response over another follows a logistic function:

$$P(y_w \succ y_l | x) = \frac{\exp(r_\phi(y_w|x))}{\exp(r_\phi(y_w|x)) + \exp(r_\phi(y_l|x))} \tag{2}$$

where $r_\phi(y|x)$ is the reward model parameterized by ϕ .

Explaining Equation (2):

This equation captures the probability of preference based on reward differences:

- $r_\phi(y|x)$: A scalar reward score for response y given prompt x , learned by a neural network
- The exponential: Ensures positive values and amplifies differences
- The denominator: Normalizes to create a valid probability distribution
- Symmetry: $P(y_w \succ y_l | x) + P(y_l \succ y_w | x) = 1$

Alternative Formulation (Sigmoid Form):

Equation (2) can be rewritten more compactly:

$$P(y_w \succ y_l | x) = \sigma(r_\phi(y_w|x) - r_\phi(y_l|x)) \tag{3}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

Why This Form is Useful:

Equation (3) makes explicit that:

- Only the *difference* in rewards matters for preferences, not absolute values
- If $r_\phi(y_w|x) - r_\phi(y_l|x) = 0$, then $P(y_w \succ y_l | x) = 0.5$ (indifferent)
- If $r_\phi(y_w|x) \gg r_\phi(y_l|x)$, then $P(y_w \succ y_l | x) \approx 1$ (strong preference)
- The sigmoid provides a smooth, differentiable transition between preference strengths

Training the Reward Model

Given a dataset of human preferences $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$, we train the reward model by maximizing the likelihood of observed preferences.

Likelihood Function:

The likelihood of the entire dataset under the Bradley-Terry model is:

$$\mathcal{L}(\phi) = \prod_{i=1}^N P(y_w^{(i)} \succ y_l^{(i)} | x^{(i)}) = \prod_{i=1}^N \frac{\exp(r_\phi(y_w^{(i)} | x^{(i)}))}{\exp(r_\phi(y_w^{(i)} | x^{(i)})) + \exp(r_\phi(y_l^{(i)} | x^{(i)}))} \quad (4)$$

Loss Function:

In practice, we minimize the negative log-likelihood:

$$\mathcal{L}(\phi) = - \sum_{i=1}^N \log P(y_w^{(i)} \succ y_l^{(i)} | x^{(i)}) \quad (5)$$

Substituting the Bradley-Terry formula:

$$\mathcal{L}(\phi) = - \sum_{i=1}^N \log \sigma(r_\phi(y_w^{(i)} | x^{(i)}) - r_\phi(y_l^{(i)} | x^{(i)})) \quad (6)$$

This is equivalent to binary cross-entropy loss, treating preference prediction as binary classification.

Explaining Equation (6):

This loss function guides reward model training:

- For each preference pair, we want $r_\phi(y_w | x) > r_\phi(y_l | x)$
- The loss is minimized when the reward difference is large and positive
- Using log-sigmoid makes the gradient well-behaved and prevents numerical instability
- The negative sign converts maximization (of likelihood) to minimization (standard in optimization)

Gradient Computation:

The gradient with respect to parameters ϕ is:

$$\nabla_\phi \mathcal{L}(\phi) = - \sum_{i=1}^N (1 - \sigma(\Delta r^{(i)})) \left(\nabla_\phi r_\phi(y_w^{(i)} | x^{(i)}) - \nabla_\phi r_\phi(y_l^{(i)} | x^{(i)}) \right) \quad (7)$$

where $\Delta r^{(i)} = r_\phi(y_w^{(i)} | x^{(i)}) - r_\phi(y_l^{(i)} | x^{(i)})$.

Gradient Interpretation:

The gradient naturally balances:

- $(1 - \sigma(\Delta r^{(i)}))$: The "mistake weight" - larger when the model is wrong
- If $\Delta r^{(i)}$ is large (model confident and correct): small gradient, little update needed
- If $\Delta r^{(i)}$ is negative (model wrong): large gradient, significant correction
- The difference of reward gradients: pushes winner's reward up and loser's reward down

RLHF Pipeline with Preference Modeling

The preference modeling stage fits into the broader RLHF pipeline as illustrated in Figure 1.

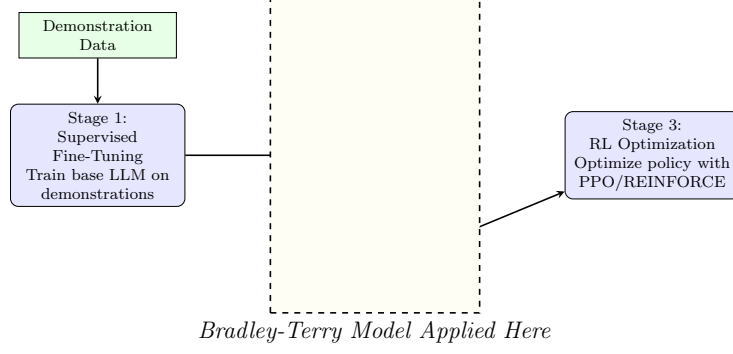


Figure 1: RLHF pipeline highlighting the preference modeling stage

Worked Example: Training a Reward Model

Let’s walk through a concrete example of training a reward model on preference data using the Bradley-Terry framework.

Setup:

- Prompt: x = "Explain photosynthesis"
- Response y_w : "Photosynthesis is the process by which plants convert light energy into chemical energy..."
- Response y_l : "Plants make food from sunlight."
- Human annotator prefers y_w (more detailed and informative)

Step 1: Initial Reward Predictions

Suppose our reward model initially assigns:

$$r_\phi(y_w|x) = 2.3 \quad (8)$$

$$r_\phi(y_l|x) = 2.8 \quad (9)$$

The model incorrectly thinks y_l is better!

Step 2: Compute Preference Probability

Using the Bradley-Terry model:

$$P(y_w \succ y_l|x) = \sigma(r_\phi(y_w|x) - r_\phi(y_l|x)) = \sigma(2.3 - 2.8) = \sigma(-0.5) \quad (10)$$

$$\sigma(-0.5) = \frac{1}{1 + e^{0.5}} = \frac{1}{1 + 1.649} \approx 0.378 \quad (11)$$

The model predicts only 37.8% probability that y_w is preferred, but the true label is 100% (annotator chose y_w).

Step 3: Compute Loss

The loss for this single example:

$$\mathcal{L} = -\log \sigma(r_\phi(y_w|x) - r_\phi(y_l|x)) = -\log(0.378) \approx 0.973 \quad (12)$$

This relatively high loss indicates the model needs significant adjustment.

Step 4: Compute Gradient Weight

The gradient weight:

$$1 - \sigma(\Delta r) = 1 - 0.378 = 0.622 \quad (13)$$

This weight determines how much to update the parameters. Since the model is quite wrong, the weight is substantial.

Step 5: Conceptual Parameter Update

The gradient update (simplified) pushes:

- $r_\phi(y_w|x)$ **upward** by $\alpha \cdot 0.622 \cdot \nabla_\phi r_\phi(y_w|x)$
- $r_\phi(y_l|x)$ **downward** by $\alpha \cdot 0.622 \cdot \nabla_\phi r_\phi(y_l|x)$

where α is the learning rate.

Step 6: After Update (Hypothetical)

After training, suppose the reward model learns:

$$r'_\phi(y_w|x) = 3.5 \quad (14)$$

$$r'_\phi(y_l|x) = 1.8 \quad (15)$$

Now the reward difference is $\Delta r = 3.5 - 1.8 = 1.7$, giving:

$$P(y_w \succ y_l|x) = \sigma(1.7) \approx 0.845 \quad (16)$$

The model now correctly assigns higher probability (84.5%) to preferring y_w .

Step 7: Loss Improvement

The new loss:

$$\mathcal{L}' = -\log(0.845) \approx 0.169 \quad (17)$$

The loss decreased from 0.973 to 0.169, confirming the model has improved on this example.

Learning Insight:

The Bradley-Terry model elegantly converts a discrete preference judgment ("I prefer A over B") into continuous, differentiable objectives. The reward model learns to assign higher scores to responses that humans prefer, creating a reliable reward signal for subsequent RL training.

Properties and Extensions of Bradley-Terry

Key Properties:

- **Transitivity:** If $P(A \succ B) > 0.5$ and $P(B \succ C) > 0.5$, then $P(A \succ C) > 0.5$

- **Scale Invariance:** Adding a constant to all rewards doesn't change preferences: $r(y) + c$ gives same $P(y_w \succ y_l)$
- **Interpretability:** Reward differences directly correspond to log-odds of preference
- **Probabilistic Calibration:** Output probabilities reflect confidence in preferences

Important Limitations:

- **Pairwise Assumption:** Only models binary comparisons, not rankings of multiple items
- **No Ties:** Standard Bradley-Terry doesn't model equal preferences (can be extended)
- **Context Independence:** Assumes preferences don't depend on what else is being compared
- **Transitivity Assumption:** Human preferences are not always transitive in practice

Practical Extensions:

- **Margin-based variants:** Add margin to enforce stronger separation: $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l) - m)$
- **Multi-response ranking:** Extend to Plackett-Luce model for full rankings
- **Confidence weighting:** Weight preferences by annotator agreement or confidence
- **Contextual rewards:** Condition reward model on additional features beyond prompt and response

Connection to RLHF Training

In RLHF, the reward model trained via Bradley-Terry becomes the objective for policy optimization:

- **Reward Signal:** The scalar $r_\phi(y|x)$ guides the RL algorithm (PPO, REINFORCE)
- **Policy Objective:** Maximize $\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[r_\phi(y|x)]$ subject to KL constraints
- **Preference Consistency:** The learned policy should generate responses that humans would prefer
- **Feedback Loop:** Better policies generate better comparison data for reward model refinement

The elegance of this approach is that we never need explicit "correct" outputs - only relative preferences. This makes RLHF scalable to complex, subjective tasks like dialogue where absolute correctness is ill-defined.

Numerical Stability Considerations

When implementing Bradley-Terry models, numerical stability is crucial:

- **LogSumExp Trick:** Compute $\log(\exp(a) + \exp(b))$ as $\max(a, b) + \log(1 + \exp(-|a - b|))$
- **Reward Normalization:** Normalize rewards to prevent extreme values: $r' = (r - \mu) / \sigma$
- **Gradient Clipping:** Prevent exploding gradients during training
- **Log-Space Operations:** Work with log-probabilities when possible to avoid under-flow

Summary

The Bradley-Terry model provides the mathematical foundation for preference-based learning in RLHF:

- Converts pairwise human preferences into a scalar reward function
- Uses a probabilistic framework that's interpretable and differentiable
- Enables learning from subjective, comparative feedback
- Bridges human judgment and RL optimization

The key insight is that relative comparisons are more reliable than absolute ratings for subjective tasks. By learning a reward model from preferences, we create a differentiable objective that captures human values, enabling language models to be optimized for alignment with human preferences.

Practical Implementation Notes:

Real-world preference modeling must address:

- **Data Quality** - ensuring consistent, high-quality preference annotations
- **Reward Hacking** - preventing models from exploiting reward model weaknesses
- **Distribution Shift** - maintaining reward model accuracy as policy improves
- **Preference Disagreement** - handling cases where annotators disagree

References

1. Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4), 324-345.
2. Christiano, P. F., et al. (2017). Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 30.
3. Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.
4. Stiennon, N., et al. (2020). Learning to summarize from human feedback. *Advances in Neural Information Processing Systems*, 33.